

AI-Assisted Software Development Internship

Weeks 1-2 | Days 1-10 | Task Report

Code, screenshots and evidence for every daily lab

Prepared by: Mirza Yasir Abdullah Baig

Role: AI Engineer

GitHub user: mirzayasirabdullahbaig07

Repository: <https://github.com/mirzayasirabdullahbaig07/AI-Internship>

Date: 10 October 2026

1. Summary of work

All ten daily labs from the handbook were completed as working code or documents in one Git repository, with one branch per task, Conventional Commit messages and a clean merge history (27 commits on `main`). Everything that could be executed here was executed; what could not be done from this environment is listed honestly in section 2.

Day	Task	Deliverable (folder)	Evidence
1	Static profile card	day-01-profile-card	Playwright: no overflow 320/1280, hover colour changes
2	Responsive sign-in (CSS Grid)	day-02-responsive-signin	Playwright: no overflow at 320/390/768/1280; Tab order; Enter submits
3	Typed task list, 4 UI states	day-03-typed-tasks	Jest: 6 tests
4	API fetch with retry	day-04-api-retry	Jest vs real local HTTP server: 7 tests
5	Password strength meter	day-05-password-strength	Jest: 11 tests; written limitations note
6	Local Git discipline	day-06-git-local	Real one-file, one-commit lab
7	GitHub and Draft PR	day-07-github-draft-pr	Ready-to-paste Draft PR body (PR not opened)
8	PR review discipline	day-08-pr-review	Failing test -> fix -> passing; PR template
9	Local test matrix	day-09-local-test-matrix	UI vs DB scenarios; formatter commit; verify run
10	CI failures and honest reporting	day-10-ci-failures	5 classified logs; honest report with SHA

2. Honest status: what was and was not done

Per the handbook's Rule 5 (honest evidence), these are the limits of this submission:

- **Not pushed to GitHub.** I have no access to your GitHub account from here. The repo is ready: run the commands in section 14.
- **Day 7 Draft PR not opened** (needs your login). The PR body is written in `DRAFT_PR.md`.
- **Hosted CI was not run.** `.github/workflows/verify.yml` exists but has never executed.
- **Day 8 review was a self-review** of my Day 4 diff because there is no partner PR here; it found and fixed one real bug.
- **Day 10 CI logs are samples I wrote**; the instructor's five official logs were not available.
- **JSONPlaceholder was not called** (no internet in the build sandbox); Day 4 tests use a real local mock server instead.
- **Screenshots** come from local headless Chromium and rendered terminal output, not a deployed staging site.

What was verified

`npm run verify` (Prettier + TypeScript + Jest) exited 0 on commit `00f1a62`: 3 suites, 24 tests passed. The browser script passed 10 of 10 checks.

3. Day 1 - Software Delivery Basics: Profile Card

A plain HTML/CSS card (name, role, Follow button), no JavaScript. The button is a real `<button>`, the page is semantic (`main / article / h1`) and the layout is fluid.

Acceptance criteria (written first)

- The Follow button has a distinct hover state (darker colour, 1px lift).
- Visible keyboard focus ring via `:focus-visible`; native button so Tab, Enter and Space work.
- Semantic markup, `lang="en"`, no horizontal scroll at 320px.

day-01-profile-card/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Profile Card - Day 1</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <main>
      <article class="card" aria-labelledby="name">
        <div class="avatar" aria-hidden="true">YB</div>
        <h1 id="name" class="name">Mirza Yasir Abdullah Baig</h1>
        <p class="role">AI Engineer</p>
        <button type="button" class="follow-btn">Follow</button>
      </article>
    </main>
  </body>
</html>
```

Desktop (1280px)



Hover state



Mobile (320px)



Screenshot: desktop, hover state (colour changed `rgb(47,75,214)` to `rgb(31,52,168)`) and 320px mobile

Knowledge check: UI change vs data change

A **UI change** (button colour) only changes presentation in the browser and needs no server. A **data change** (saving the follow state) crosses the trust boundary: the browser sends a request, the API checks authorization and validates it, the database persists it, and the response updates the UI. It needs auth, validation, error handling and tests, and must survive a refresh.

4. Day 2 - HTML, CSS and Responsive UI: Sign-in Form

Email + Password form using CSS Grid, mobile-first (320px base), with breakpoints at 390, 768 and 1024px. Labels are linked with `label` for; grid columns use `minmax(0, 1fr)` and inputs `min-width: 0` to avoid the implicit min-content overflow (the grid/flex equivalent of the handbook's gotcha).

day-02-responsive-signin/style.css

```
/* Mobile-first: one column at 320px */
.page {
  display: grid;
  grid-template-columns: minmax(
    0,
    1fr
  ); /* minmax(0,...) prevents grid blowout (like flex min-width:0) */
  gap: 24px;
  max-width: 960px;
  margin: 0 auto;
  padding: 24px 16px;
}
.intro h1 {
  margin: 0 0 8px;
  font-size: 1.5rem;
}
.intro p {
  margin: 0;
  color: var(--muted);
}
.signin {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  gap: 16px;
  padding: 24px 16px;
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 6px 20px rgb(27 34 64 / 10%);
}
.field {
  display: grid;
  gap: 6px;
  min-width: 0;
}
label {
  font-weight: 600;
}
input {
  width: 100%;
  min-width: 0;
  font: inherit;
  padding: 12px;
  border: 1.5px solid var(--border);
  border-radius: 8px;
}
input:focus-visible,
.submit:focus-visible {
  outline: 3px solid #ffb703;
  outline-offset: 2px;
}
.submit {
  font: inherit;
  font-weight: 700;
  padding: 12px;
  border: 0;
  border-radius: 8px;
  background: var(--brand);
  color: #fff;
  cursor: pointer;
}
.submit:hover {
  background: #1f34a8;
}

/* Large mobile: a bit more breathing room */
@media (min-width: 390px) {
  .signin {
    padding: 28px 24px;
  }
}

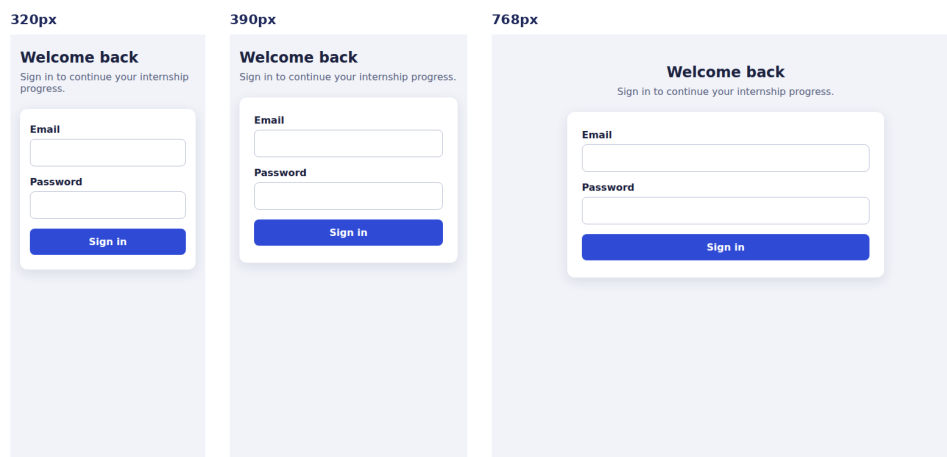
/* Tablet: centred narrow form */
```

```

@media (min-width: 768px) {
  .page {
    padding: 48px 32px;
  }
  .signin {
    max-width: 520px;
    width: 100%;
    margin: 0 auto;
  }
  .intro {
    text-align: center;
  }
}

/* Desktop: two-column grid (intro | form) */
@media (min-width: 1024px) {
  .page {
    grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
    align-items: center;
    min-height: 100vh;
  }
  .intro {
    text-align: left;
  }
  .intro h1 {
    font-size: 2.25rem;
  }
  .signin {
    margin: 0;
  }
}

```



Screenshot: the form at 320px, 390px and 768px

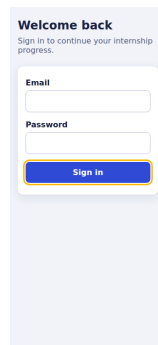
Automated checks (Playwright)

Check	Result
Horizontal overflow at 320 / 390 / 768 / 1280px	0px at every width - PASS
Keyboard-only Tab order	email, password, button - PASS
Enter on focused button submits the form	PASS
Every input has a matching label	PASS

```
terminal - browser checks

$ python3 tools/verify_day02.py
[Day 1] width=320px horizontal overflow=0px -> PASS
[Day 1] width=1280px horizontal overflow=0px -> PASS
[Day 1] hover state: rgb(47, 75, 214) -> rgb(31, 52, 168) -> PASS
[Day 2] width=320px horizontal overflow=0px -> PASS
[Day 2] width=390px horizontal overflow=0px -> PASS
[Day 2] width=768px horizontal overflow=0px -> PASS
[Day 2] width=1280px horizontal overflow=0px -> PASS
[Day 2] Tab order: ['email', 'password', 'BUTTON'] -> PASS
[Day 2] Enter on focused button submits form -> PASS
[Day 2] every input has a <label for> -> PASS
```

Screenshot: output of tools/verify_day02.py



Screenshot: keyboard focus ring on the Sign in button at 320px

5. Day 3 - TypeScript Essentials: Typed Task List

A raw task array was converted into typed data handling. `UIState` is a discriminated union with **four** states (the handbook snippet lists three; *empty* is added because it is a distinct reality). `fakeFetchTasks` simulates latency/failure with `async/await`; `loadTasks` converts any outcome into a state and never throws; `describeState` uses an exhaustive switch so the compiler flags any missed case.

day-03-typed-tasks/tasks.ts

```
export interface Task {
  id: string;
  title: string;
  completedAt: Date | null; // null = explicitly NOT completed
}

/** Every data fetch has four UI states: loading, empty, error, success. */
export type UIState =
  | { status: "loading" }
  | { status: "empty" }
  | { status: "error"; message: string }
  | { status: "success"; data: Task[] };

export const initialState: UIState = { status: "loading" };

export const sampleTasks: Task[] = [
  {
    id: "t1",
    title: "Write profile card",
    completedAt: new Date("2026-10-01T10:00:00Z"),
  },
  { id: "t2", title: "Make sign-in responsive", completedAt: null },
  { id: "t3", title: "Type the task list", completedAt: null },
];
```

Knowledge check: null vs undefined vs []

- **null** - the value exists and is intentionally 'nothing' (task not completed).
- **undefined** - never set or not initialised (not fetched yet, or a bug).
- **[]** - fetched successfully, zero records: show *Empty*, not an error or a spinner.

Result: 6 Jest tests cover loading, success, empty, error, non-Error throws, and the null/undefined/[] distinction - all pass (see section 12).

6. Day 4 - APIs and Data Boundaries: Fetch with Retry

A `fetchWithRetry` wrapper that sends a bearer token, retries network drops and 5xx with exponential backoff, and does **not** retry 4xx (a bad request or missing token can never succeed on retry). Tests run against a real local HTTP server that destroys the first two connections to simulate network drops.

day-04-api-retry/fetchWithRetry.ts

```
/** The server answered 200 but the body was not valid JSON - retrying will not help. */
export class InvalidResponseError extends Error {
  constructor(message = "Invalid JSON in response body") {
    super(message);
    this.name = "InvalidResponseError";
  }
}

export interface FetchOptions {
  token: string;
  retries?: number; // extra attempts after the first
  baseDelayMs?: number; // exponential backoff base
  fetchImpl?: typeof fetch; // injectable for tests
}

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

/**
 * Fetch JSON with a bearer token and retry on transient failures
 * (network drops and 5xx). Client errors (4xx, incl. 401/403) are NOT retried:
 * retrying a bad request or missing token can never succeed.
 */
export async function fetchWithRetry<T>(url: string, opts: FetchOptions): Promise<T> {
  const { token, retries = 2, baseDelayMs = 100, fetchImpl = fetch } = opts;
  for (let attempt = 0; ; attempt++) {
    try {
      const res = await fetchImpl(url, { headers: { Authorization: `Bearer ${token}` } });
      if (res.status === 401) throw new HttpError(401, "Auth required");
      if (!res.ok) throw new HttpError(res.status, `HTTP error: ${res.status}`);
      try {
        return (await res.json()) as T;
      } catch {
        throw new InvalidResponseError();
      }
    } catch (err) {
      const retryable =
        err instanceof InvalidResponseError
          ? false
          : !(err instanceof HttpError) || err.status >= 500;
      if (!retryable || attempt >= retries) throw err;
      await sleep(baseDelayMs * 2 ** attempt);
    }
  }
}
```

Knowledge check: three cases where the client must not be trusted

- **Cart totals / prices** - users can edit the request; the server recomputes from its own price table.
- **Admin access** - hiding a button or sending `isAdmin:true` proves nothing; the server checks the role from the verified token every request.
- **Tenant data** - a client can change `?tenantId=`; the server derives the tenant from the authenticated identity.

7. Day 5 - Vibe Coding with Critical Thinking: Password Strength Meter

Built from a bounded prompt (Goal / Scope / Must preserve / Acceptance checks / Do not / Return - recorded in the Day 5 README). Then I deliberately attacked the result, including pasting 10,000 characters: input is truncated to 128 characters for analysis, the user is told, and it runs in under 50 ms (tested).

day-05-password-strength/passwordStrength.ts

```
export function evaluatePassword(input: string): Strength {
  if (input.length === 0) {
    return {
      score: 0,
      label: "Empty",
      truncated: false,
      feedback: ["Enter a password."],
    };
  }
  const truncated = input.length > MAX_LEN;
  const pw = truncated ? input.slice(0, MAX_LEN) : input;
  const feedback: string[] = [];
  if (truncated) feedback.push(`Only the first ${MAX_LEN} characters are checked.`);

  if (pw.length < 8) {
    feedback.push("Use at least 8 characters.");
    return { score: 0, label: "Too short", truncated, feedback };
  }

  let points = 0;
  if (pw.length >= 8) points++;
  if (pw.length >= 12) points++;
  if (pw.length >= 16) points++;

  const classes = [/[a-z]/, /[A-Z]/, /[0-9]/, /^[A-Za-z0-9]/].filter((re) =>
    re.test(pw),
  ).length;
  points += classes - 1; // 1 class = 0 extra, 4 classes = +3
  if (classes < 3) feedback.push("Mix upper/lower case, numbers and symbols.");

  if (COMMON.has(pw.toLowerCase())) {
    feedback.push("This is a very common password.");
    points = 0;
  }
  if (/^(.)\1+$/ .test(pw)) {
    feedback.push("Avoid repeating a single character.");
    points = 0;
  }

  const score = Math.max(1, Math.min(4, Math.floor(points / 1.5) + 1)) as Score;
  const final: Score = points === 0 ? 1 : score;
  return { score: final, label: LABELS[final], truncated, feedback };
}
```

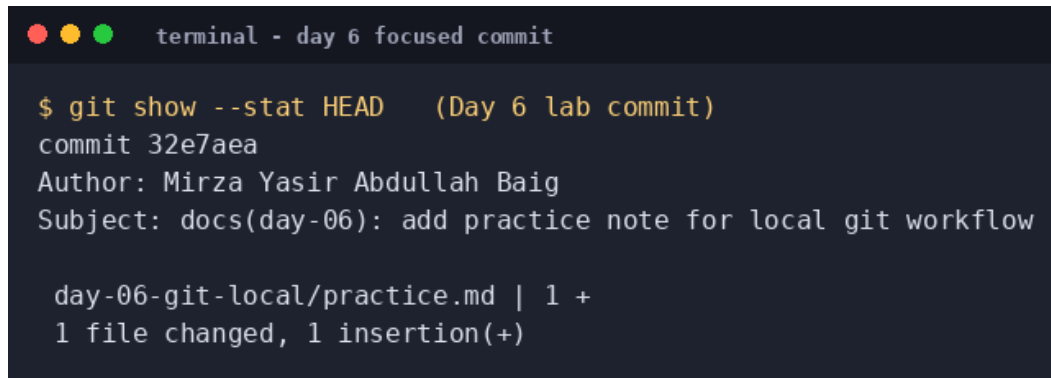
"Why this could be wrong" note (verified by probing the real function)

- **Leetspeak scores 'Good':** P@ssw0rd and Abcdefg1 both score 3/4 though they are in every cracking dictionary - the AI assumed variety equals strength.
- The common-password list has only 12 entries; a real check uses millions (e.g. zxcvbn).
- Keyboard walks are not detected: qwertyuiopas scores Fair.
- Non-ASCII letters (ñ, ü) are counted as 'symbols' - the AI assumed English/ASCII input.
- Truncation at 128 was a silent product decision. Strength rules must also be enforced server-side.

These are recorded as known limitations, not fixed - finding them was the lab goal.

8. Day 6 - Git Locally

Lab: create a branch, modify one file, make one focused, correctly formatted commit with zero unrelated files. Branch feat/day-06-git-local modified only day-06-git-local/practice.md; git show --stat lists exactly one file.

A terminal window with a dark background and light-colored text. The title bar reads 'terminal - day 6 focused commit'. The command '\$ git show --stat HEAD (Day 6 lab commit)' is entered. The output shows the commit hash '32e7aea', the author 'Mirza Yasir Abdullah Baig', and the subject 'docs(day-06): add practice note for local git workflow'. Below this, it shows 'day-06-git-local/practice.md | 1 +' and '1 file changed, 1 insertion(+)'.

Screenshot: the Day 6 commit touches exactly one file

Workflow used: git switch -c (one task = one branch), git status, git add <specific file> (never git add .), git diff --cached --stat, then a type(scope): summary commit message.

9. Day 7 - GitHub and the Draft PR

GitHub is the system of record: branch protection, PR discussion, issues and an audit log. The branch feat/day-07-draft-pr contains the Draft PR body (purpose, scope, screenshots, known limitations, reviewer) and exact push/PR commands. **The PR itself is not opened** because it requires your GitHub login.

day-07-github-draft-pr/DRAFT_PR.md

Purpose

Complete the Day 7 lab: push a feature branch to GitHub and open a **Draft** PR with purpose, scope, screenshots and known limitations.

Scope

- Adds `day-07-github-draft-pr/` documentation only.
- No application code changed. No dependencies added.

Screenshots

- Day 1 profile card: `report/screenshots/day01\_1280.png`
- Day 2 sign-in at 320px: `report/screenshots/day02\_320.png`
- Local verification output: `report/screenshots/verify\_output.png`

Known limitations

- Draft status: not ready for merge until a human reviewer approves.
- Hosted CI has not been run on this branch.
- Screenshots are generated by local Playwright runs, not from a deployed staging site.

Reviewer

Assign a human reviewer. **Do not self-merge** (Non-negotiable Rule 4).

10. Day 8 - PR Review and Merge Discipline

I reviewed the diff (not the author) against scope, security, tests and edge cases. Finding: if a server answers **200** with a non-JSON body, `res.json()` throws and my catch treated it as a network error and **retried** identical bad requests. I proved it with a failing test first, then fixed it with a non-retryable `InvalidResponseError`.

```
terminal - day 8 failing test then fix

=== BEFORE FIX ===
✓ recovers after 2 simulated network drops (3rd attempt succeeds) (47 ms)
✓ sends the Authorization bearer header (16 ms)
✓ retries 5xx then succeeds (9 ms)
✓ 401 is NOT retried and surfaces 'Auth required' (11 ms)
✓ gives up after retries are exhausted (1 + 2 attempts) on persistent 500 (27 ms)
✓ gives up on persistent network drops (35 ms)
✗ 200 with an invalid JSON body is NOT retried (retrying cannot fix a bad body) (26 ms)
  - Expected   - 3
  + Received   + 1
    74 |   expect(hits).toBe(1);
Tests:    1 failed, 6 passed, 7 total
=== AFTER FIX ===
✓ recovers after 2 simulated network drops (3rd attempt succeeds) (50 ms)
✓ sends the Authorization bearer header (12 ms)
✓ retries 5xx then succeeds (10 ms)
✓ 401 is NOT retried and surfaces 'Auth required' (7 ms)
✓ gives up after retries are exhausted (1 + 2 attempts) on persistent 500 (25 ms)
✓ gives up on persistent network drops (40 ms)
✓ 200 with an invalid JSON body is NOT retried (retrying cannot fix a bad body) (7 ms)
Tests:    7 passed, 7 total
```

Screenshot: before fix 1 failed / 6 passed, after fix 7 passed

The mandatory PR template (Problem solved, Files changed, Risk / permissions, Local checks run, Known limits, Final SHA) was added at `.github/pull_request_template.md`. *Note: this was a self-review because no partner PR was available; do a real peer review on GitHub as well.*

11. Day 9 - Local-first CI Without Waste

The test matrix compares a UI button change (format, types, component test, focused browser check) with a database migration (real Postgres in Docker, up/down migration, integration, contract and tenant-isolation tests). The DB scenario is theoretical for this repo. Prettier was introduced in its own style: commit to keep formatting separate from logic. Final Candidate Rule applied: one verify suite on one exact SHA.

Change type	Local evidence before push
UI button	format:check, typecheck, component test, browser check, git diff --stat
Database migration	docker compose db, migrate up/down, integration, contract, tenant isolation, verify

12. Test results

```
terminal - jest (all suites)

$ npx jest --verbose
PASS day-05-password-strength/passwordStrength.test.tsx
  evaluatePassword
    ✓ empty string (3 ms)
    ✓ short passwords are 'Too short'
    ✓ common password is capped at Weak
    ✓ single repeated character is Weak
    ✓ long mixed password is Strong (1 ms)
    ✓ longer is not worse: passphrase beats short complex (1 ms)
    ✓ 10,000-char paste is truncated, flagged, and fast (1 ms)
    ✓ emoji / unicode do not throw (1 ms)
    ✓ spaces count as symbols (passphrase friendly)
  <PasswordStrengthMeter />
    ✓ updates the meter and live text as the user types (83 ms)
    ✓ pasting 10,000 chars does not crash the component (10 ms)
PASS day-04-api-retry/fetchWithRetry.test.ts
  ✓ recovers after 2 simulated network drops (3rd attempt succeeds) (51 ms)
  ✓ sends the Authorization bearer header (15 ms)
  ✓ retries 5xx then succeeds (8 ms)
  ✓ 401 is NOT retried and surfaces 'Auth required' (3 ms)
  ✓ gives up after retries are exhausted (1 + 2 attempts) on persistent 500 (32 ms)
  ✓ gives up on persistent network drops (41 ms)
  ✓ 200 with an invalid JSON body is NOT retried (retrying cannot fix a bad body) (7 ms)
PASS day-03-typed-tasks/tasks.test.ts
  loadTasks - the four UI states
    ✓ starts in loading (1 ms)
    ✓ success: returns the tasks (5 ms)
    ✓ empty: request succeeded but 0 records (9 ms)
    ✓ error: network failure is captured, not thrown (2 ms)
    ✓ error: non-Error throw still produces a safe message (1 ms)
  null vs undefined vs []
    ✓ they are three different realities (1 ms)
Test Suites: 3 passed, 3 total
Tests:      24 passed, 24 total
Snapshots:  0 total
Time:       1.902 s
Ran all test suites.
```

Screenshot: jest --verbose, all suites

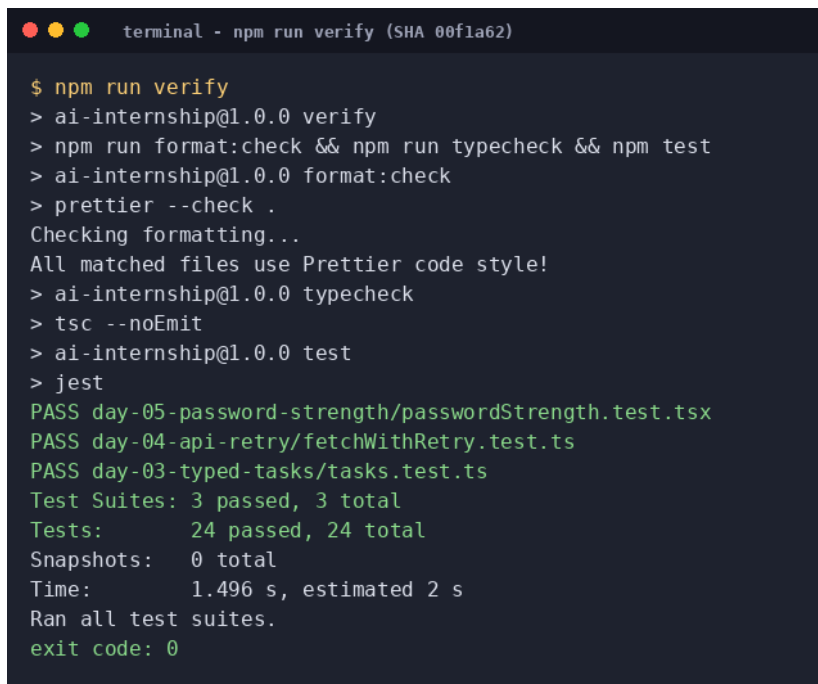
13. Day 10 - CI Failures and Honest Reporting

Five sample CI logs were classified (the instructor's official logs were not available, so these are my own practice samples):

Log	Class	Evidence	Honest status
1	Code	toHaveStyle assertion failed	Failed: 1 test
2	Baseline (flaky)	Same test fails on main, README-only change	Failed on baseline too
3	Infra	Docker Hub toomanyrequests, 0 steps ran	Not Run
4	Code	TypeScript error TS2339	Failed: type error
5	Infra	Billing limit, 0 of 6 steps ran	Not Run

Honest report for this repository

Field	Value
Command run	npm run verify (prettier --check, tsc --noEmit, jest)
Result	Exit code 0. 3 suites, 24 tests passed. Browser script: 10/10 checks passed.
Final candidate SHA	00f1a62d9fe1a5d3e4eb81a5e15b36753cbf8502
Limitations	Hosted CI NOT run; Draft PR not opened; no peer review; no JSONPlaceholder call; samples not official logs.



```
terminal - npm run verify (SHA 00f1a62)

$ npm run verify
> ai-internship@1.0.0 verify
> npm run format:check && npm run typecheck && npm test
> ai-internship@1.0.0 format:check
> prettier --check .
Checking formatting...
All matched files use Prettier code style!
> ai-internship@1.0.0 typecheck
> tsc --noEmit
> ai-internship@1.0.0 test
> jest
PASS day-05-password-strength/passwordStrength.test.tsx
PASS day-04-api-retry/fetchWithRetry.test.ts
PASS day-03-typed-tasks/tasks.test.ts
Test Suites: 3 passed, 3 total
Tests:      24 passed, 24 total
Snapshots:  0 total
Time:       1.496 s, estimated 2 s
Ran all test suites.
exit code: 0
```

Screenshot: npm run verify on the final candidate commit

Later commits (Day 10 notes, README, this report) are documentation only; no source file changed after the verified SHA.

14. Git history and how to push

```
terminal - git history

$ git log --graph --oneline --decorate -n 32
* e2b84b1 (HEAD -> main) Merge docs/day-10-ci-failures
|\
| * 35413ea (docs/day-10-ci-failures) docs(day-10): classify sample CI logs and add honest report
|/
* 00f1a62 Merge docs/day-09-local-test-matrix
|\
| * e748053 (docs/day-09-local-test-matrix) style: format repository with prettier
| * b486cec docs(day-09): add local test matrix for UI change vs DB migration
|/
* 98cfd14 Merge fix/day-08-review-findings
|\
| * b7738cd (fix/day-08-review-findings) fix(day-04): do not retry when a 200 response has an inval...
|/
* 4928630 Merge feat/day-07-draft-pr
|\
| * 8a14f8a (feat/day-07-draft-pr) docs(day-07): add draft PR instructions and body
|/
* c386f0d Merge feat/day-06-git-local
|\
| * 9dbba42 (feat/day-06-git-local) docs(day-06): document disciplined git workflow
| * 32e7aea docs(day-06): add practice note for local git workflow
|/
* 4b494e1 Merge feat/day-05-password-strength
|\
| * 7ca6776 (feat/day-05-password-strength) feat(day-05): add password strength meter with edge-cas...
|/
* 4195aa9 Merge feat/day-04-api-retry
|\
| * d645e85 (feat/day-04-api-retry) feat(day-04): add fetchWithRetry with backoff and local mock-se...
|/
* c3ffc9c Merge feat/day-03-typed-tasks
|\
| * 9e7dfb0 (feat/day-03-typed-tasks) feat(day-03): add typed task list with four explicit UI states
|/
* 4dbbba3 Merge test/day-02-browser-checks
|\
| * cald00e (test/day-02-browser-checks) test(day-02): add Playwright overflow and keyboard checks ...
|/
* 0b7b4ce Merge feat/day-02-responsive-signin
|\
| * ed611d8 (feat/day-02-responsive-signin) feat(day-02): add responsive sign-in form with CSS grid
|/
* 2fe120f Merge feat/day-01-profile-card
|\
| * 5240656 (feat/day-01-profile-card) feat(day-01): add static profile card with hover and focus s...
|/
* d9cb474 chore: initial project scaffold and tooling
```

Screenshot: one branch per task, merged with --no-ff (history shown up to the Day 10 merge)

Push commands (run on your machine, inside the unzipped repo folder)

```
git remote add origin https://github.com/mirzayasirabdullahbaig07/AI-Internship.git
git push -u origin main
git push origin --all
# then open the Day 7 Draft PR (see day-07-github-draft-pr/README.md)
```

Note on workflow: I merged the per-day branches locally so main contains everything. In a team repo with branch protection, you would instead push each branch and open a PR that a human reviews - never self-merge (Rule 4).

Mirza Yasir Abdullah Baig

AI Engineer | github.com/mirzayasirabdullahbaig07